

Identificação de Exoplanetas Utilizando Técnicas de Aprendizado Profundo

Gabriel Marochi Schmidt¹, Carolina Paula de Almeida¹, Sandra Mara Guse Scós Venske¹

¹Departamento de Ciência da Computação (DECOMP) – Universidade Estadual do Centro-Oeste (UNICENTRO)
Guarapuava – PR – Brasil

`gabrielmarochischmidt@gmail.com, {carol,ssvenske}@unicentro.br`

Abstract. *The identification of exoplanets, planets that orbit stars outside the Solar System, has become a relevant field of research, driven by space missions such as Kepler and TESS. Traditionally, this identification requires manual analysis by experts, which is a time-consuming and resource-intensive process. Computational techniques, especially those related to Deep Learning, have been utilized in the literature to address this problem. Given this scenario, this work proposes the implementation of Deep Learning techniques for the automatic detection of exoplanets from the analysis of light curves available in the TESS mission archives. To this end, in-depth studies of the involved techniques were conducted, as well as their adaptation and implementation for exoplanet detection, with the support of computational tools and specialized libraries. In the end, three models - CNN, InceptionTime, and Mamba - were obtained and their performances were compared. Through this, it was observed that the new approaches, not yet tested in this scope, are very promising and keep pace with the performance of robust techniques in this field. The contribution of this work is the study of promising techniques in an area where their performance had not yet been tested.*

Keywords: *Convolutional Neural Network, InceptionTime, Mamba, Light Curves.*

Resumo. *A identificação de exoplanetas, planetas que orbitam estrelas fora do Sistema Solar, tornou-se um campo de pesquisa relevante, impulsionado por missões espaciais como a Kepler e TESS. Tradicionalmente, essa identificação exige análise manual por especialistas, o que demanda tempo e recursos. Técnicas computacionais, especialmente relacionadas ao Aprendizado Profundo, têm sido utilizadas na literatura para abordar esse problema. Diante desse cenário, este trabalho propõe a implementação de técnicas de Aprendizado Profundo para a detecção automática de exoplanetas a partir da análise de curvas de luz disponíveis nos arquivos da missão TESS. Para isso, foram realizados estudos aprofundados sobre as técnicas envolvidas, bem como sua adaptação e implementação para a detecção de exoplanetas, com o apoio de ferramentas computacionais e bibliotecas especializadas. Ao final, três modelos - CNN, InceptionTime e Mamba - foram obtidos e seus desempenhos comparados, com isso, observou-se que as novas abordagens, ainda não testadas*

nesse escopo, são muito promissoras e acompanham o desempenho de técnicas robustas nesse tema. A contribuição deste trabalho é o estudo de técnicas promissoras em uma área onde seu desempenho ainda não havia sido testado.

Palavras-Chave: Rede Neural Convolucional, InceptionTime, Mamba, Curvas de Luz.

1. Introdução

No campo da Astronomia, a busca por exoplanetas — planetas que orbitam estrelas fora do nosso Sistema Solar — tem ganhado destaque na comunidade científica, especialmente após o lançamento das missões espaciais Kepler e TESS (*Transiting Exoplanet Survey Satellite*), cujos principais objetivos incluíam a detecção desses corpos celestes [Shallue and Vanderburg 2018].

Tradicionalmente, a identificação de exoplanetas pode ser realizada por meio de diversas técnicas, como o método da velocidade radial, astrometria, imagem direta e *gravitational microlensing*. Ainda assim, a maioria dos exoplanetas conhecidos foi descoberta utilizando o método do trânsito planetário. Isso se deve à sua alta eficiência para missões de observação em larga escala, como a TESS, já que permite monitorar milhares de estrelas simultaneamente em busca de variações periódicas de brilho compatíveis com o trânsito de um planeta [Borucki et al. 2010].

Esse método baseia-se na observação do brilho de uma estrela ao longo do tempo. Quando um planeta passa em frente à sua estrela hospedeira (do ponto de vista do observador), parte da luz é bloqueada, provocando uma leve queda na intensidade do brilho registrado — a chamada curva de luz [Winn 2010]. A profundidade e a duração desse decréscimo dependem dos tamanhos relativos do planeta e da estrela. Entretanto, esse método apresenta algumas limitações: os trânsitos só podem ser detectados quando a órbita do planeta está alinhada com a linha de visão do observador, e há uma alta taxa de falsos positivos, como sistemas binários eclipsantes ou trânsitos de estrelas com dimensões semelhantes às de planetas. Além disso, a grande quantidade de dados gerados por missões como a Kepler dificulta a análise manual e aumenta a necessidade de triagem automatizada. Por isso, é necessário confirmar as descobertas com métodos complementares, o que torna o processo de validação dispendioso e moroso [Malik et al. 2022].

Com o avanço da Inteligência Artificial (IA) e do Aprendizado de Máquina (AM), diversas técnicas vêm sendo aplicadas neste contexto com o intuito de automatizar a detecção de exoplanetas, reduzindo o tempo necessário para a análise e aumentando a taxa de descobertas [Jara-Maldonado et al. 2020, Shallue and Vanderburg 2018, Malik et al. 2022].

Do ponto de vista computacional, a curva de luz de uma estrela é uma série temporal, ou seja, uma sequência de pontos de dados medidos em intervalos de tempo sucessivos. A tarefa de identificar um trânsito planetário, portanto, se enquadra em um problema de classificação de séries temporais, onde o objetivo é analisar a sequência e determinar se ela contém o padrão característico de um exoplaneta.

Neste contexto, este trabalho propõe a aplicação de diferentes técnicas de Aprendizado Profundo para a identificação de exoplanetas a partir da análise das curvas de luz. Foram utilizados dados disponibilizados publicamente pelos autores do trabalho *Identifying Exoplanets with Deep Learning. V. Improved Light-curve Classification for TESS*

*Full-frame Image Observations*¹ [Tey et al. 2022], além de ferramentas computacionais especializadas para o pré-processamento, treinamento, validação e teste do modelo. Para isso, foram estudadas e aplicadas diversas ferramentas disponíveis na linguagem *Python*², como as bibliotecas *scikit-learn*³, *TensorFlow*⁴ e *PyTorch*⁵, que facilitam a implementação e experimentação de modelos de Aprendizado de Máquina. A escolha dessas ferramentas deve-se à sua ampla adoção na comunidade científica e à robustez dos recursos oferecidos para tarefas supervisionadas de classificação.

As técnicas implementadas e avaliadas neste trabalho foram uma Rede Neural Convolutacional (CNN) de uma dimensão, com arquitetura baseada no trabalho de Christopher J. Shallue e Andrew Vanderburg [Shallue and Vanderburg 2018], uma rede neural especializada em séries temporais chamada InceptionTime [Ismail Fawaz et al. 2020], e uma nova técnica baseada em Modelos de Espaço Estruturados (*Selective State Spaces*, SSMs) chamada Mamba, que tem apresentado um desempenho muito bom em problemas que lidam com sequências muito longas [Gu and Dao 2023].

Ao final, embora as três abordagens tenham alcançado um desempenho similar na métrica principal de Average Precision (em torno de 0,85), a análise detalhada revelou especialidades distintas para cada arquitetura. A InceptionTime, por exemplo, obteve a maior precisão (0,90), porém em detrimento da sensibilidade, ou *recall* (0,67). A Mamba confirmou sua vantagem teórica, apresentando de longe o menor tempo de treinamento (24% e 7% do tempo que a CNN e InceptionTime tiveram, respectivamente). A CNN, por sua vez, se destacou como o modelo mais equilibrado, alcançando o melhor resultado na Average Precision (0,875), o que demonstra o potencial promissor das novas abordagens quando comparadas a um forte padrão de referência.

O presente trabalho se estrutura da seguinte forma: inicialmente, na Seção 2, são introduzidos os principais conceitos referentes à identificação de exoplanetas e Aprendizado Profundo. Em seguida, na Seção 3, é detalhado o processo de desenvolvimento dos modelos e técnicas aplicados ao problema, bem como a preparação dos dados e os métodos de treinamento e validação. Finalmente, nas Seções 4 e 5 são apresentados os resultados e as considerações finais sobre o trabalho.

2. Fundamentação Teórica

Nesta seção são apresentados os conceitos fundamentais que sustentam esse trabalho. A subseção 2.1 aborda e contextualiza os conceitos relacionados aos exoplanetas, bem como seus métodos de identificação. Na subseção 2.2 são apresentados os conceitos relacionados ao Aprendizado Profundo e suas técnicas. A subseção 2.3 discute as métricas utilizadas para avaliação dos experimentos. Por fim, a subseção 2.4 aborda os trabalhos correlatos considerados.

2.1. Identificação de Exoplanetas

Desde as primeiras descobertas planetárias, a área da detecção de exoplanetas tem sido um tópico relevante na comunidade científica [Malik et al. 2022], principalmente após

¹<https://zenodo.org/records/7411579>

²<https://docs.python.org/3/>

³<https://scikit-learn.org>

⁴<https://www.tensorflow.org>

⁵<https://pytorch.org>

as missões espaciais Kepler [Borucki et al. 2010] e TESS [Ricker et al. 2014], que criaram um vasto catálogo de sistemas exoplanetários. O estudo desses sistemas é de grande importância na Astronomia moderna, pois contribui para a compreensão da formação do nosso Sistema Solar e da possibilidade de vida em outros locais do universo [Jara-Maldonado et al. 2020].

Formalmente, um exoplaneta, ou planeta extrassolar, é definido como um objeto com massa menor do que 13 vezes a massa de Júpiter, que orbita uma estrela e foi encontrado fora do Sistema Solar [Khan et al. 2017, Jara-Maldonado et al. 2020]. Desde as suas primeiras descobertas [Wolszczan and Frail 1992], milhares desses mundos já foram encontrados por meio de diversos métodos distintos, dentre os quais se destacam [Jara-Maldonado et al. 2020]:

- **Velocidade Radial:** Detecta exoplanetas por meio das variações no espectro da estrela causadas pela influência gravitacional do planeta, observando o efeito Doppler. Essa técnica permite identificar planetas pela oscilação da estrela ao redor do centro de massa do sistema.
- **Imagem Direta:** Busca capturar imagens do exoplaneta separando-o espacialmente da luz da estrela hospedeira, permitindo analisar características como composição química e temperatura. Essa técnica exige instrumentos avançados, como coronógrafos, devido à proximidade e ao brilho intenso das estrelas em relação aos exoplanetas.
- **Microlente Gravitacional:** Detecta exoplanetas observando o aumento temporário do brilho de uma estrela distante, causado pela distorção da luz por objetos massivos, como planetas, ao passarem em frente a essa estrela. Essa técnica depende de alinhamentos raros e imprevisíveis entre os objetos.

Apesar do avanço e da variedade de técnicas disponíveis, cada método apresenta desafios próprios. A imagem direta, por exemplo, exige instrumentos sofisticados para diferenciar o brilho do exoplaneta da estrela hospedeira, enquanto a microlente gravitacional depende de alinhamentos celestes raros. A técnica de velocidade radial, embora robusta, também tem limitações, pois as oscilações estelares causadas por exoplanetas são muito pequenas, o que dificulta sua detecção.

Diante dessas limitações, o método do trânsito planetário surge como a alternativa mais prolífica e utilizada nessa área [Winn 2010]. O método consiste em monitorar o brilho de uma estrela ao longo do tempo, buscando pequenas quedas periódicas na luminosidade causadas pela passagem de um exoplaneta em frente à estrela, do ponto de vista do observador [Charbonneau et al. 2000, Borucki et al. 2010]. Entre suas principais vantagens estão a possibilidade de observar múltiplos sistemas estelares simultaneamente e a detecção de planetas de diferentes tamanhos. No entanto, o método depende de um alinhamento específico entre a órbita do planeta e a linha de visão do observador, o que limita a fração de exoplanetas detectáveis por essa técnica [Malik et al. 2022].

2.2. Aprendizado Profundo

O Aprendizado de Máquina (AM) é um subcampo da Inteligência Artificial (IA) que se concentra no desenvolvimento de algoritmos que permitem aos sistemas computacionais aprender a partir de dados, sem serem explicitamente programados para cada tarefa específica [Samuel 1959, Mitchell 1997]. Enquanto o AM clássico se destaca em problemas

descritos por regras formais, o verdadeiro desafio da IA moderna reside na resolução de tarefas intuitivas para humanos, como o reconhecimento de padrões em imagens, áudio ou, no contexto deste trabalho, em séries temporais [Goodfellow et al. 2016].

Para solucionar esses problemas mais complexos, surge o Aprendizado Profundo, uma subárea especializada do AM. A abordagem do subcampo é permitir que os computadores aprendam com a experiência e compreendam o mundo por meio de uma hierarquia de conceitos, onde representações mais abstratas e complexas são construídas a partir de outras mais simples. Essa estrutura hierárquica resulta em modelos com múltiplas camadas de processamento, formando uma arquitetura “profunda”, daí a origem do termo [Goodfellow et al. 2016].

Nas subseções seguintes, serão detalhadas as arquiteturas de Aprendizado Profundo utilizadas neste trabalho.

2.2.1. Redes Neurais Convolucionais

Dentro do Aprendizado Profundo, as Redes Neurais Convolucionais (*Convolutional Neural Network*, CNN) são uma classe de redes neurais profundas que ganharam muita notoriedade devido ao seu desempenho formidável em tarefas envolvendo imagens e visão computacional [Lecun et al. 2015]. Elas são inspiradas na neurociência visual e têm esse nome pois aplicam filtros (*kernels*) a uma entrada (imagem ou, no contexto desse trabalho, séries temporais) por meio da convolução a fim de extrair características relevantes [Elgendy 2020].

A arquitetura típica de uma CNN é composta por uma sequência de camadas. A camada convolucional é o bloco central, onde os *kernels* são aplicados na entrada realizando operações de convolução para gerar mapas de características. Esses filtros são especializados em detectar padrões locais, como bordas em imagens ou, no caso de dados de uma dimensão (1D), segmentos específicos de uma série temporal [Goodfellow et al. 2016].

Após as camadas convolucionais, geralmente são aplicadas funções de ativação não lineares, como a ReLU, para introduzir não-linearidade ao modelo. Existe também a camada de *pooling*, que tem como principal função reduzir a dimensionalidade dos mapas de características, tornando a representação mais compacta e conferindo um grau de invariância a pequenas translações nos dados de entrada. Finalmente, após várias camadas convolucionais e de *pooling*, uma ou mais camadas totalmente conectadas são tipicamente usadas para realizar a tarefa de classificação com base nas características de alto nível extraídas [Goodfellow et al. 2016].

No contexto da classificação de exoplanetas a partir de curvas de luz, as CNNs representam o estado da arte consolidado. O trabalho de [Shallue and Vanderburg 2018], que introduziu a arquitetura AstroNet, foi pioneiro ao estabelecer a eficiência dessa rede para este problema. As otimizações e evoluções subsequentes propostas pelo mesmo grupo de pesquisa, como as de [Yu et al. 2019, Tey et al. 2023], reforçam a posição das CNNs como referência na área, justificando sua implementação neste trabalho como base de comparação para outras arquiteturas.

2.2.2. InceptionTime

A InceptionTime é uma arquitetura de rede neural profunda, introduzida por [Ismail Fawaz et al. 2020], projetada especificamente para alcançar alta precisão em tarefas de classificação de séries temporais. A arquitetura adapta para o domínio temporal o conceito dos módulos *Inception*, que foram inicialmente popularizados em modelos de visão computacional de grande sucesso, como o *GoogLeNet* [Szegedy et al. 2015].

O principal diferencial da InceptionTime reside em seu bloco de construção fundamental, o módulo *Inception*. Diferente de uma CNN padrão, que tipicamente aplica filtros de um único tamanho por camada, este módulo utiliza um conjunto de filtros convolucionais com tamanhos de *kernel* variados que operam em paralelo sobre a mesma sequência de entrada [Ismail Fawaz et al. 2020]. Essa abordagem paralela permite que a rede aprenda a extrair características de diferentes granularidades simultaneamente: *kernels* menores capturam padrões de curta duração, enquanto *kernels* maiores identificam tendências de longo prazo [Ismail Fawaz et al. 2020].

Para manter a eficiência computacional, a arquitetura também emprega camadas de “gargalo” (*bottleneck*) para reduzir a dimensionalidade dos dados antes das convoluções mais custosas, uma técnica herdada do módulo *Inception* original [Szegedy et al. 2015, Ismail Fawaz et al. 2020]. As saídas de todos os filtros paralelos são então concatenadas e uma rede InceptionTime completa é formada pelo empilhamento de vários desses módulos, geralmente interligados por conexões residuais para facilitar o treinamento de modelos mais profundos [Ismail Fawaz et al. 2020].

Dado o fato de que as CNNs constituem o estado da arte na área da classificação de exoplanetas, uma investigação natural é avaliar se arquiteturas convolucionais mais especializadas podem oferecer um desempenho ainda superior. A InceptionTime se encaixa nesse cenário, pois é uma variante de CNN projetada especificamente para a análise de séries temporais. Sua capacidade de analisar padrões em múltiplas escalas simultaneamente representa uma evolução teórica sobre as CNNs padrão, com isso, a análise e comparação desta arquitetura se tornam muito pertinentes.

2.2.3. Mamba

O Mamba é uma arquitetura de rede neural recente, baseada em Modelos de Espaço de Estados Estruturados (SSMs), que se destaca na resolução de problemas nas áreas de linguagem, áudio e genômica, principalmente quando se trata de problemas com entradas muito longas. Ele surge como uma alternativa promissora em relação a modelos do estado da arte, como o Transformer, nesses domínios onde o contexto de longo alcance é crucial e a complexidade quadrática dos mecanismos de atenção se torna um gargalo computacional. O Mamba, por outro lado, alcança alto desempenho enquanto escala linearmente com o tamanho da sequência [Gu and Dao 2023].

A ideia central do Mamba é aprimorar os SSMs clássicos por meio de um mecanismo de seleção. Enquanto SSMs tradicionais gerenciam a evolução do estado com matrizes fixas (A, B, C, D), no Mamba esses parâmetros são gerados a partir da própria entrada. Isso introduz a seletividade que permite ao modelo decidir se deve propagar

informações importantes ou esquecer informações irrelevantes ao longo da sequência, adaptando-se dinamicamente ao contexto [Gu and Dao 2023].

A unidade fundamental da arquitetura é o Bloco Mamba. Ele processa os dados através de dois caminhos paralelos: o principal utiliza uma convolução 1D para capturar padrões locais, seguida pelo núcleo SSM para modelar eficientemente as dependências de longo alcance. O segundo caminho funciona como um portão seletivo (*gating mechanism*), cujo resultado é combinado com o do caminho principal através de uma multiplicação elemento a elemento. Esse mecanismo de portão permite que o modelo controle dinamicamente qual informação deve ser propagada pela rede, selecionando as características mais relevantes da sequência em cada passo [Gu and Dao 2023].

A inclusão da arquitetura Mamba nesta pesquisa foi motivada pela exploração de um paradigma fundamentalmente diferente das abordagens convolucionais. Sua complexidade linear permite um treinamento e inferência significativamente mais rápidos em comparação com outras arquiteturas complexas. Portanto, sua avaliação neste trabalho visa investigar como um modelo de estado da arte, baseado em SSMs e com alta eficiência, se comporta nesta tarefa de classificação, oferecendo uma alternativa inovadora ao domínio das convoluções.

2.3. Métricas de Avaliação

Neste trabalho foram utilizadas cinco métricas para avaliar o desempenho dos modelos. Essas métricas são Precisão, *Recall*, F_1 -score, *Average Precision* e área sob a Curva Característica de Operação do Receptor (AUC-ROC). Tais métricas são detalhadas nas Subseções 2.3.1, 2.3.2 e 2.3.3.

As métricas consideradas utilizam as seguintes medições:

- VP são as instâncias classificadas como *Verdadeiros Positivos*;
- VN são as instâncias classificadas como *Verdadeiros Negativos*;
- FP são as instâncias classificadas como *Falsos Positivos*; e
- FN são as instâncias classificadas como *Falsos Negativos*.

2.3.1. Métrica F_1 -score

A métrica F_1 -score combina as métricas de precisão (Equação 1) e sensibilidade (*recall*) (Equação 2) em uma única métrica, calculando a média harmônica entre elas. É particularmente útil quando há um desbalanceamento entre as classes, como é o caso do problema abordado neste trabalho. A Equação 3 define a métrica F_1 [Shalev-Shwartz and Ben-David 2014].

$$Precisão = \frac{VP}{VP + FP} \quad (1)$$

$$Sensibilidade = \frac{VP}{VP + FN} \quad (2)$$

$$F_1 = 2 \times \frac{Precisão \times Sensibilidade}{Precisão + Sensibilidade} = \frac{VP}{VP + \frac{FN+FP}{2}} \quad (3)$$

2.3.2. Métrica *Average Precision*

A *Average Precision* (AP) é uma métrica que resume a curva de Precisão-Sensibilidade (*Precision-Recall curve*, PR) em um único valor. A curva PR plota a precisão *versus* a sensibilidade para diferentes limiares de classificação. A AP é calculada como a média ponderada das precisões obtidas em cada limiar, sendo que o peso é o aumento na sensibilidade em relação ao limiar anterior [Manning et al. 2008]. A *Average Precision* é especialmente informativa em tarefas com grande desbalanceamento de classes, pois foca no desempenho da classe positiva. A Equação 4 define a AP, sendo que P_n e R_n são a precisão e a sensibilidade no n -ésimo limiar.

$$AP = \sum_n (R_n - R_{n-1}) P_n \quad (4)$$

2.3.3. Métrica AUC-ROC

A métrica AUC-ROC corresponde à área sob a Curva Característica de Operação do Receptor (*AUC - Receiver Operating Characteristic*). A curva ROC ilustra o desempenho de um classificador binário ao variar o limiar de decisão, plotando a Taxa de Verdadeiros Positivos (TVP) contra a Taxa de Falsos Positivos (TFP) [Fawcett 2006].

A TVP é equivalente à Sensibilidade (Equação 2), enquanto a TFP (Equação 5) mede a proporção de negativos que foram incorretamente classificados como positivos.

$$TFP = \frac{FP}{FP + VN} \quad (5)$$

A AUC pode ser interpretada como a probabilidade de que um exemplo positivo, escolhido aleatoriamente, seja classificado com uma pontuação maior do que um exemplo negativo, também escolhido aleatoriamente [Fawcett 2006].

2.4. Trabalhos Correlatos

Esta seção revisa os estudos fundamentais que se relacionam com o presente trabalho. A análise se concentra em duas vertentes principais: abordagens de AM já consolidadas na classificação de exoplanetas e arquiteturas inovadoras para a análise de séries temporais que motivam a abordagem proposta nesta pesquisa.

Um trabalho pioneiro na área é o de Christopher J. Shallue e Andrew Vanderburg [Shallue and Vanderburg 2018] que introduziu uma Rede Neural Convolutacional (CNN) chamada AstroNet para a classificação binária de curvas de luz da missão Kepler [Borucki et al. 2010]. O modelo e a metodologia proposta foram utilizados como base para a implementação das técnicas desenvolvidas neste trabalho.

Dando continuidade ao trabalho de [Shallue and Vanderburg 2018], o estudo de Evan Tey [Tey et al. 2023] apresentou o AstroNet-Triage-v2, uma evolução do modelo original proposto anteriormente pelo mesmo grupo [Yu et al. 2019], que introduziu uma série de processos, como o uso de múltiplas visões da curva de luz e a inclusão de metadados tabulares para melhorar o desempenho da classificação nos dados da missão TESS.

O artigo também produziu e disponibilizou publicamente o conjunto de dados utilizado para treinar e avaliar o modelo, o qual foi adotado como o principal conjunto de dados para os experimentos desta pesquisa.

Embora as abordagens baseadas em Redes Neurais Convolucionais representem o estado da arte para este problema, diversas pesquisas, como [Malik et al. 2022], vêm investigando a performance de outros paradigmas e arquiteturas na área. Nesse contexto, este trabalho investigou e avaliou tanto técnicas clássicas quanto técnicas promissoras, porém ainda não testadas, no escopo do problema.

[Malik et al. 2022] propôs uma abordagem diferente da convencional da área, na qual os autores utilizaram a biblioteca *tsfresh* [Christ et al. 2018] para extrair as características das séries temporais e utilizar esses dados como entrada em um modelo *LightGBM*. Essa técnica, baseada em *gradient boosting*, constrói árvores de decisão de forma sequencial, onde cada nova árvore corrige os erros da anterior. O *LightGBM* é conhecido por sua alta performance e eficiência, representando uma alternativa moderna e robusta para tarefas de classificação [Ke et al. 2017].

Outra arquitetura de relevância para este estudo é a *InceptionTime*, proposta pelo estudo de Ismail Fawaz e outros colaboradores [Ismail Fawaz et al. 2020]. O modelo adapta os bem-sucedidos módulos *Inception*, da área de visão computacional, para a análise de séries temporais. Sua principal característica é a aplicação de múltiplos filtros convolucionais de diferentes tamanhos em paralelo, permitindo que a rede capture padrões em diversas escalas temporais simultaneamente. Dada sua eficácia comprovada em problemas de classificação de séries temporais, a *InceptionTime* foi incluída neste trabalho como uma abordagem convolucional especializada.

Finalmente, um desenvolvimento recente que motiva a contribuição central deste trabalho é a arquitetura *Mamba*, introduzida por [Gu and Dao 2023]. Baseada em Modelos de Espaço de Estados (SSMs), *Mamba* alcançou desempenho de estado da arte em domínios como processamento de linguagem, áudio e genômica, destacando-se pela sua eficiência em modelar sequências longas. O presente trabalho propõe utilizar e avaliar essa arquitetura inovadora no contexto da classificação de exoplanetas.

3. Materiais e Métodos

Nesta seção é apresentada a metodologia utilizada neste trabalho, bem como a organização geral da implementação das técnicas de Aprendizado Profundo. O repositório online referente a este trabalho pode ser acessado via *Github*⁶.

3.1. Metodologia Proposta

A Figura 1 descreve toda a metodologia adotada neste trabalho e as seções 3.1.1, 3.1.2 e 3.1.3 detalham cada uma dessas fases.

3.1.1. Obtenção e Pré-Processamento dos Dados

A etapa inicial da metodologia consistiu na obtenção do conjunto de dados para o treinamento e validação dos modelos. Para este fim, foi utilizado o catálogo fornecido publica-

⁶<https://github.com/GabrielMSchmidt/code-tcc>

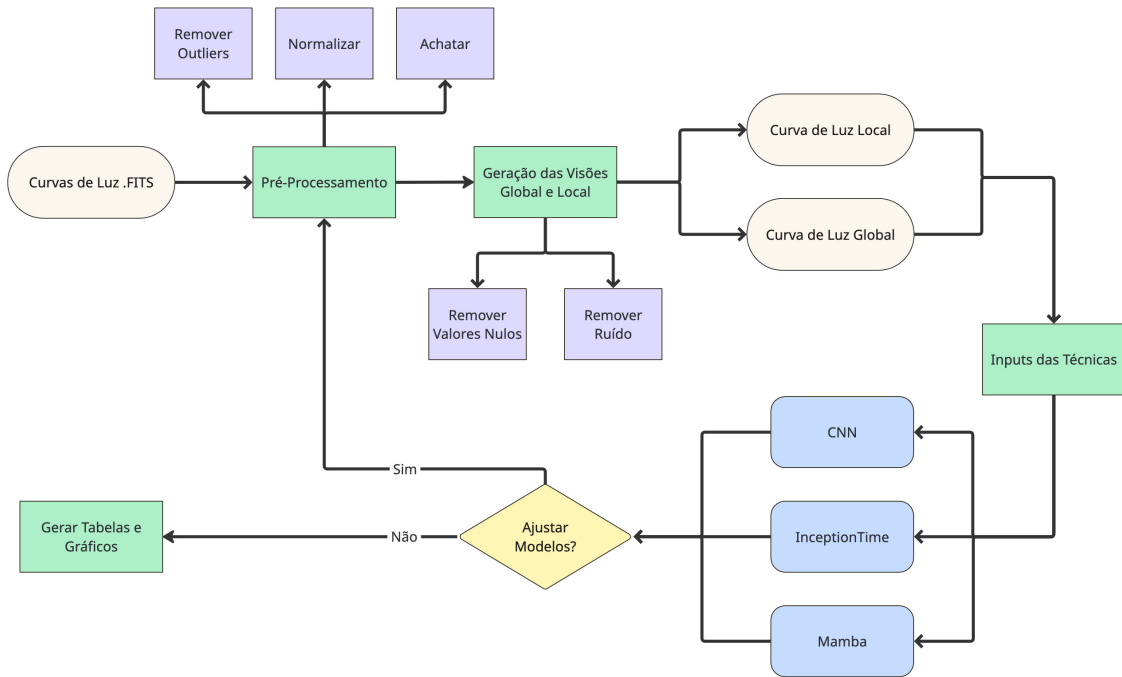


Figura 1. Diagrama Geral da Metodologia Proposta. O fluxograma ilustra as etapas sequenciais do trabalho, partindo da obtenção e pré-processamento dos dados. Em seguida, são geradas as visões local e global das curvas de luz, que servem de entrada para as três arquiteturas de Aprendizado Profundo avaliadas (CNN, InceptionTime e Mamba). Por fim, o ciclo de ajuste e teste dos modelos leva à geração dos resultados finais. Fonte: O Autor.

mente pelo estudo de [Tey et al. 2023], que contém aproximadamente 25.000 curvas de luz de estrelas observadas pela missão TESS, previamente classificadas e validadas.

Uma vez adquirido, o conjunto de dados brutos passou por uma série de etapas de pré-processamento, seguindo a metodologia proposta em [Shallue and Vanderburg 2018], com a finalidade de normalizar e limpar os dados para realçar os sinais de trânsito planetário. Todas essas operações foram implementadas com o auxílio da biblioteca Lightkurve [Lightkurve Collaboration et al. 2018], uma ferramenta especializada na análise de dados fotométricos. As seguintes técnicas foram aplicadas:

- **Remoção de Outliers:** Pontos de dados anômalos, que se desviam significativamente do fluxo luminoso médio, foram removidos utilizando um algoritmo de *sigma clipping* [Astropy Collaboration et al. 2013]. Este método iterativo descarta medições que excedem um determinado número de desvios-padrão da mediana local.
- **Normalização:** O fluxo de cada curva de luz foi normalizado para que a mediana fosse zero e o valor mínimo -1. Este procedimento padroniza a profundidade dos trânsitos em todas as instâncias, garantindo que os modelos não sejam enviesados pela magnitude do brilho estelar [Shallue and Vanderburg 2018].
- **Achatamento (Flattening):** Variações de baixa frequência no brilho estelar, geralmente causadas pela própria atividade da estrela ou por artefatos instrumentais, foram removidas. Para isso, aplicou-se um filtro de *Savitzky-Golay* [Savitzky and Golay 1964] para subtrair essa tendência da curva de luz, isolando

os eventos de trânsito.

3.1.2. Geração das Visões

Após essas operações, duas representações, ou visões, da curva de luz são criadas: uma visão local e uma visão global. Esta estratégia permite que os modelos analisem simultaneamente o comportamento detalhado do trânsito planetário (visão local) e seu contexto dentro do período de observação (visão global), melhorando a capacidade de distinção de sinais verdadeiros [Shallue and Vanderburg 2018, Jara-Maldonado et al. 2020].

Ainda seguindo a metodologia de [Shallue and Vanderburg 2018], para refinar ainda mais estas visões, foram aplicados os seguintes tratamentos:

- **Remover Valores Nulos:** Lacunas nos dados, resultantes de interrupções na observação, foram preenchidas por meio de interpolação, utilizando a média dos pontos vizinhos.
- **Remover Ruído (*Binning*):** A técnica de *binning* foi empregada para suavizar as visões. Este processo consiste em agrupar pontos de dados sequenciais em intervalos de tempo e substituí-los por sua média, o que aumenta a razão sinal-ruído do trânsito. A técnica foi implementada seguindo a mesma metodologia descrita em [Shallue and Vanderburg 2018] e sua definição formal também pode ser encontrada no referido trabalho.

Essas duas visões foram utilizadas como entrada para as três arquiteturas de Aprendizado Profundo avaliadas neste trabalho: CNN, InceptionTime e Mamba.

3.1.3. Ajuste dos Modelos e Geração dos Resultados

Durante a implementação e teste dos modelos, diversas combinações de conjuntos de dados, hiperparâmetros, arquiteturas e técnicas de pré-processamento foram testadas até os modelos novos atingirem um desempenho próximo ao da CNN, arquitetura base utilizada neste projeto.

Após isso, a etapa final da metodologia compreendeu a avaliação do desempenho dos modelos otimizados. Utilizando um conjunto de teste independente, foram calculadas métricas quantitativas, como Precisão, *Recall*, F_1 -score, *Average Precision* e AUC-ROC. Os resultados foram então compilados e visualizados por meio de matrizes de confusão, gráficos de curvas ROC e tabelas comparativas.

3.2. Conjunto de Dados

O conjunto de dados utilizado foi obtido por meio do trabalho [Tey et al. 2023], no qual as curvas de luz foram extraídas do *TESS Quick-Look Pipeline* (QLP) [Kunimoto et al. 2021] e rotuladas manualmente por sete especialistas durante o período de mais de dois anos.

Esse conjunto de dados possui três versões distintas, que foram criadas pelos autores para lidar com as diferentes escalas de tempo da variabilidade estelar, fornecendo assim uma visão mais completa das características do ruído de cada curva de luz. Para gerar cada versão, foi aplicado um processo de remoção de tendência (*detrending*)

com diferentes configurações de espaçamento. A variante `SAP_FLUX_LAG`, que utiliza um espaçamento maior otimizado para preservar sinais de trânsitos de longa duração [Tey et al. 2023], foi a selecionada para os experimentos deste trabalho após uma fase de testes preliminares, nos quais a variante mencionada obteve o melhor desempenho geral.

O conjunto de dados selecionado é composto por um total de 23.913 instâncias após a fase de pré-processamento. Este conjunto apresenta um desbalanceamento de classes significativo, com 4.207 instâncias positivas (17,59%) e 19.706 instâncias negativas (82,41%).

Para o treinamento e avaliação do modelo, o conjunto total dos dados foi dividido em três subconjuntos: treinamento, validação e teste. Essa divisão foi feita de forma estratificada, a fim de manter a proporção de classes original, e seguindo a mesma divisão feita em [Tey et al. 2023]. Esses subconjuntos ficaram da seguinte forma:

- **Treinamento:** 79,91% do total, correspondendo a 19.108 instâncias.
- **Validação:** 9,95% do total, correspondendo a 2.380 instâncias.
- **Teste:** 10,14% do total, correspondendo a 2.425 instâncias.

O conjunto de treinamento foi utilizado para o aprendizado dos parâmetros dos modelos; o de validação, para o ajuste dos seguintes *callbacks*:

- ***EarlyStopping***⁷: Interrompe o treinamento quando a métrica monitorada para de melhorar.
- ***ReduceLROnPlateau***⁸: Reduz o *learning-rate* quando a métrica monitorada para de melhorar.

O conjunto de teste, por sua vez, foi utilizado para a avaliação final do desempenho dos modelos em dados não vistos.

3.3. Ferramentas e Tecnologias

Neste trabalho, foram utilizadas as seguintes ferramentas e tecnologias para alcançar os resultados finais:

- **Linguagem de Programação:** *Python*;
- **Frameworks e Bibliotecas:** *PyTorch*, *TensorFlow*, *scikit-learn*, *Keras*⁹, *sktime*¹⁰, *Mamba*¹¹, *pandas*¹², *NumPy*¹³, *Matplotlib*¹⁴, *seaborn*¹⁵, *Lightkurve*¹⁶ e *Astropy*¹⁷;
- **Ambiente de desenvolvimento:** *Visual Studio Code (VSCode)*¹⁸ e *PyCharm*¹⁹.

⁷https://keras.io/api/callbacks/early_stopping/

⁸https://keras.io/api/callbacks/reduce_lr_on_plateau/

⁹<https://keras.io/>

¹⁰<https://www.sktime.net>

¹¹<https://github.com/state-spaces/mamba>

¹²<https://pandas.pydata.org/>

¹³<https://numpy.org/>

¹⁴<https://matplotlib.org/>

¹⁵<https://seaborn.pydata.org/>

¹⁶<https://lightkurve.github.io/lightkurve/>

¹⁷<https://docs.astropy.org/>

¹⁸<https://code.visualstudio.com/Docs>

¹⁹<https://www.jetbrains.com/help/pycharm/>

A configuração de hardware utilizada foi: sistema operacional Linux, processador Intel(R) Core(TM) i5-11500 (2.7 GHz), placa gráfica NVIDIA GeForce RTX 3060 com 12 GB de memória dedicada (VRAM), além de 8 GB de memória compartilhada, e 16GB de memória RAM.

3.4. Arquitetura dos Modelos

Nesta seção, são detalhadas as configurações específicas das arquiteturas de Aprendizado Profundo implementadas.

3.4.1. CNN

A Figura 2 ilustra a arquitetura da CNN. A rede processa as visões global e local em dois ramos convolucionais paralelos e independentes, extraindo características de cada entrada. As saídas desses ramos são então concatenadas e processadas por uma sequência de camadas totalmente conectadas (FC) que realizam a classificação final.

A estrutura desta rede replica a configuração proposta em [Shallue and Vanderburg 2018], uma referência na área de identificação de exoplanetas. A principal adaptação realizada foi no tamanho dos vetores de entrada, que foram fixados em 201 para a visão global e 61 para a visão local. Essa modificação alinha a abordagem proposta com a utilizada em [Tey et al. 2023], que otimizou os tamanhos de entrada especificamente para os dados da missão TESS, os mesmos utilizados no presente trabalho.

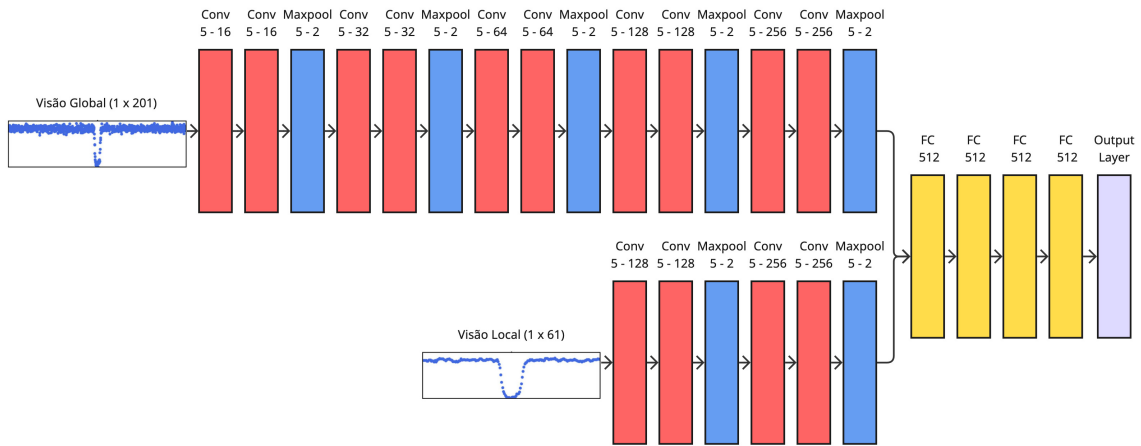


Figura 2. Diagrama da arquitetura da CNN. Camadas convolucionais são denotadas Conv [tamanho do kernel] - [número de filtros], camadas de *Max Pooling* são denotadas Maxpool [tamanho da janela] - [tamanho do passo], e camadas totalmente conectadas são denotadas FC - [número de unidades]. Fonte: Adaptado de [Shallue and Vanderburg 2018].

3.4.2. InceptionTime

A arquitetura InceptionTime foi implementada utilizando a sua versão padrão disponível na biblioteca *sktime* [Löning et al. 2019], uma ferramenta robusta para tarefas

de classificação de séries temporais. Com exceção dos parâmetros de treinamento comuns a todos os modelos (como o número de épocas e os *callbacks*), os hiperparâmetros internos da arquitetura foram mantidos em seus valores padrão, conforme recomendado pela própria biblioteca.

A implementação da InceptionTime na *sktime* aceita como entrada uma única série temporal, que pode ser univariada ou multivariada. A fim de fornecer ao modelo a informação contextual de ambas as visões (global e local) simultaneamente, foi adotada uma estratégia de entrada multivariada. A visão local, com 61 pontos, foi estendida para 201 pontos através da técnica de *padding*, que consiste em adicionar valores nulos (zeros) ao final da sequência. Em seguida, as duas visões de mesmo comprimento foram empilhadas, criando uma única série temporal de duas dimensões (formato 201x2) como entrada para o modelo.

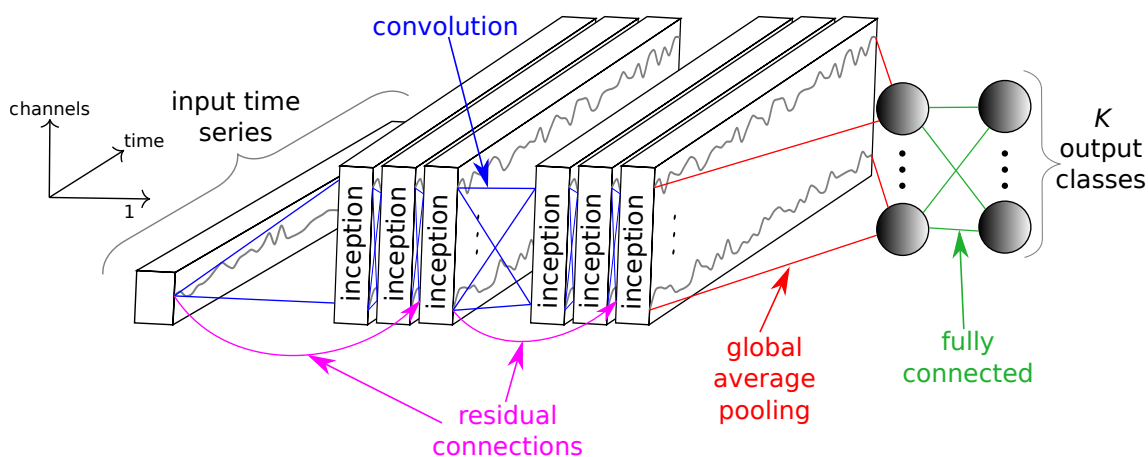


Figura 3. Estrutura completa do modelo InceptionTime, que é composta por uma sequência de blocos *Inception* empilhados. O modelo utiliza conexões residuais (*residual connections*) para ligar a entrada de um conjunto de blocos à sua saída. Ao final, uma camada de *Global Average Pooling* agrega as características aprendidas, que são passadas para uma camada totalmente conectada (*fully connected*) para a classificação final. Fonte: [Ismail Fawaz et al. 2020].

3.4.3. Mamba

A implementação do modelo Mamba neste trabalho seguiu a estrutura do Bloco Mamba descrita na Fundamentação Teórica (Seção 2.2.3). A Figura 4 ilustra a arquitetura de alto nível deste bloco, que combina projeções lineares, uma convolução 1D e o núcleo SSM, controlado por um portão multiplicativo para o fluxo de informação.

Para o treinamento da rede, foram definidos os seguintes hiperparâmetros específicos desta arquitetura, além dos hiperparâmetros comuns citados anteriormente, selecionados com base na literatura e em experimentação preliminar:

- **Taxa de aprendizado (*learning rate*):** 0,001
- **Dimensão do modelo (*d_model*):** 128
- **Dimensão do estado SSM (*d_state*):** 8
- **Dimensão da convolução (*d_conv*):** 4
- **Fator de expansão (*expand*):** 2

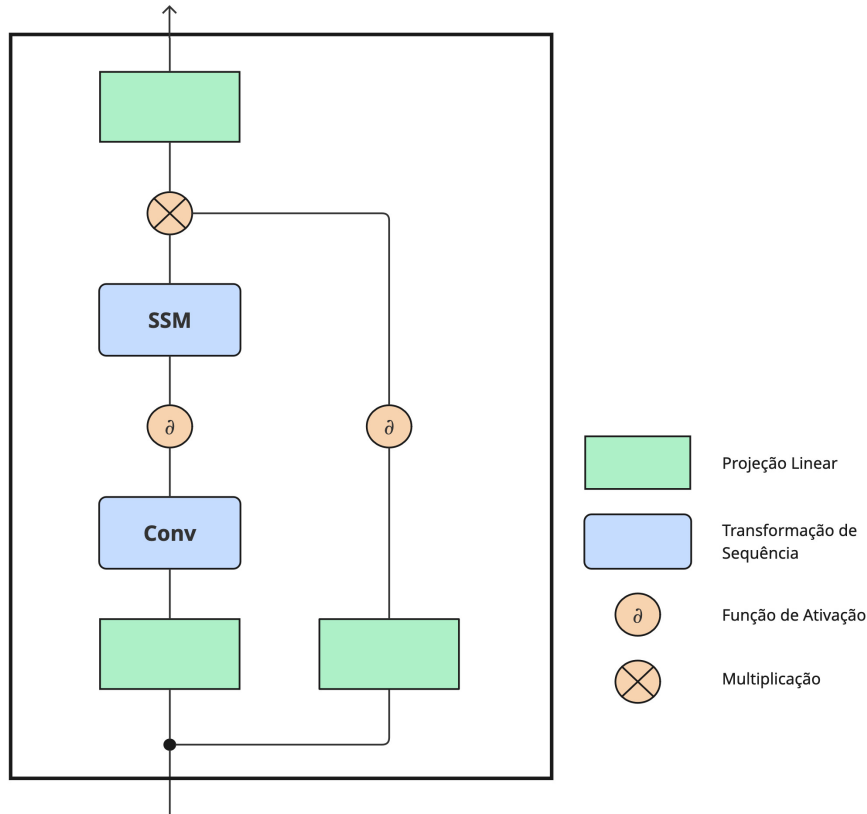


Figura 4. Diagrama da arquitetura do Bloco Mamba. A figura detalha o fluxo de dados interno do bloco. A entrada é processada em dois caminhos paralelos: um principal, que combina uma convolução (Conv) com um Modelo de Espaço de Estados (SSM), e um secundário, que atua como portão seletivo. A multiplicação dos dois caminhos permite que o modelo filtre dinamicamente a informação antes da projeção linear de saída. Fonte: O Autor, adaptado de [Gu and Dao 2023].

4. Experimentos e Resultados

A presente seção destina-se à apresentação e análise comparativa dos resultados obtidos nos experimentos com as três arquiteturas de Aprendizado Profundo propostas: CNN, InceptionTime e Mamba. A seguir, são descritas as configurações experimentais e, posteriormente, é realizada uma análise detalhada do desempenho dos modelos.

Todos os experimentos foram executados utilizando a versão `SAP_FLUX_LAG` do conjunto de dados. Os modelos foram treinados por um máximo de 200 épocas, com um critério de parada antecipada (*EarlyStopping*) que monitora a métrica de *Average Precision* no conjunto de validação. O treinamento foi interrompido caso não houvesse melhora na métrica por 25 épocas consecutivas, sendo retidos os pesos do modelo que alcançou o melhor desempenho.

Essa configuração tem como base os valores padrão estabelecidos na biblioteca utilizada, com pequenas alterações para que o critério de parada represente um equilíbrio para acomodar as diferentes velocidades de convergência dos modelos e garantir uma comparação justa.

4.1. Análise Comparativa dos Modelos

Para determinar a arquitetura mais eficaz, os resultados dos três modelos no conjunto de teste foram comparados diretamente. A Tabela 1 sintetiza essa comparação, possuindo como colunas as métricas Precisão, *Recall*, F_1 -score, *Average Precision* e AUC-ROC, e como linhas os modelos propostos.

Tabela 1. Comparação de desempenho dos modelos no conjunto de teste. Os melhores resultados para cada métrica estão destacados em uma escala de cinza, sendo o mais escuro o melhor.

Modelo	Precisão	<i>Recall</i>	F_1 -score	AP	AUC-ROC	Tempo
CNN	0,792	0,846	0,818	0,875	0,951	8m57s
InceptionTime	0,895	0,673	0,768	0,864	0,954	31m23s
Mamba	0,827	0,676	0,744	0,825	0,944	2m10s

4.1.1. Análise de F_1 -score e *Average Precision* (AP)

Analisando as métricas principais para classificação em dados desbalanceados, a CNN demonstrou uma clara superioridade, alcançando os maiores valores tanto em F_1 -score (0,818) quanto em *Average Precision* (0,875). Este resultado indica que a CNN foi a arquitetura mais bem-sucedida em manter um alto equilíbrio entre precisão e sensibilidade, sendo a mais eficaz em classificar corretamente os candidatos a exoplanetas no geral. Os modelos InceptionTime e Mamba apresentaram um desempenho ligeiramente inferior nessas métricas consolidadas.

4.1.2. Análise de Precisão, *Recall* e AUC-ROC

A análise das métricas secundárias revela diferentes comportamentos entre os modelos. A InceptionTime obteve a maior precisão (0,895), indicando a menor taxa de falsos positivos. Contudo, isso ocorreu em detrimento do *recall* (0,673), o mais baixo entre os três, sugerindo que o modelo falhou em identificar uma porção significativa dos planetas reais. A CNN, por sua vez, apresentou o melhor balanço, com o maior *recall* (0,846). Em relação à métrica AUC-ROC, todas as arquiteturas demonstraram uma excelente e muito similar capacidade de discriminação, com valores próximos a 0,95.

4.1.3. Análise de Custo Computacional

Considerando o custo computacional, o modelo Mamba confirmou sua vantagem teórica, demonstrando uma eficiência notavelmente superior com o menor tempo de treinamento (130 segundos). Em termos práticos, o Mamba foi aproximadamente 76% mais rápido que a arquitetura CNN (537 segundos) e concluiu seu treinamento em apenas 7% do tempo exigido pela InceptionTime. Em contraste, a InceptionTime revelou-se a arquitetura mais custosa, com um tempo de treinamento (1883 segundos) cerca de 250% maior que o da CNN. Essa disparidade ressalta que, para aplicações onde a velocidade de iteração e o custo de treinamento são fatores críticos, a abordagem baseada em SSMs do Mamba oferece uma vantagem prática substancial.

4.1.4. Síntese e Melhor Modelo Geral

Em síntese, embora cada arquitetura tenha apresentado pontos fortes distintos — a InceptionTime na precisão, e o Mamba na eficiência computacional — a CNN se destacou como o modelo mais equilibrado e de melhor desempenho geral. Sua superioridade nas métricas F_1 -score e AP, combinada com a maior taxa de *Recall*, a estabelece como a abordagem mais robusta e eficaz para a classificação de exoplanetas no contexto deste estudo.

A Figura 5, que ilustra o desempenho comparativo dos modelos segundo as curvas *Precision-Recall* e ROC, sintetiza o domínio da CNN, onde suas curvas demonstram uma clara superioridade perante as outras arquiteturas.

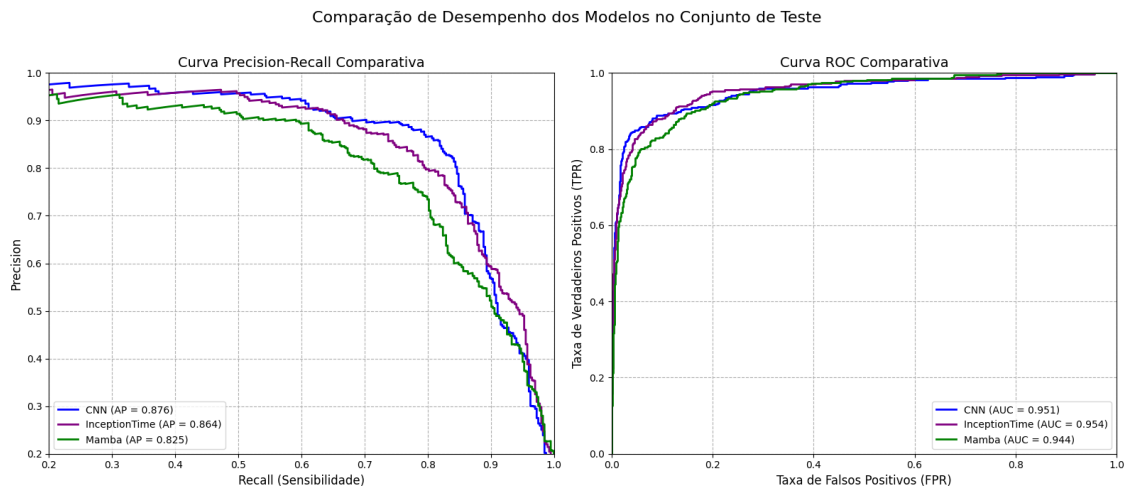


Figura 5. Desempenho dos três modelos propostos considerando as curvas *Precision-Recall* e ROC.

5. Considerações Finais

Neste trabalho, três distintas arquiteturas de Aprendizado Profundo - uma CNN tradicional, a rede InceptionTime, especializada em séries temporais, e a promissora arquitetura Mamba, baseada em SSMs - foram estudadas e comparadas para a tarefa de identificação de exoplanetas. Um esforço significativo foi dedicado à metodologia, envolvendo um rigoroso processo de obtenção, pré-processamento e preparação dos dados derivados da missão TESS, a fim de criar um *pipeline* robusto para a avaliação dos modelos.

Os resultados demonstraram que a arquitetura CNN, que serviu como padrão de referência, alcançou o melhor desempenho geral, com superioridade nas métricas de F_1 -score (0,818) e *Average Precision* (0,875). A InceptionTime obteve a maior precisão individual (0,895), enquanto o Mamba se destacou como a arquitetura mais eficiente em termos de custo computacional.

A análise foi conduzida sob a perspectiva de utilizar a CNN, um modelo de estado da arte na área, como base para avaliar o potencial de abordagens mais recentes. Embora a CNN tenha se mostrado superior no resultado consolidado, o desempenho da InceptionTime e do Mamba é considerado promissor. O fato de que essas arquiteturas, com

otimização de hiperparâmetros baseada nos valores padrão, alcançaram um desempenho próximo ao do modelo de referência já estabelecido, sugere um grande potencial. Isso indica que, com ajustes mais finos, elas podem se tornar alternativas altamente competitivas.

A principal contribuição deste trabalho, portanto, é a introdução e avaliação de arquiteturas recentes e especializadas no contexto da classificação de exoplanetas, fornecendo um *benchmark* inicial que aponta para novas direções de pesquisa. Demonstrou-se que tanto uma CNN especializada em séries temporais quanto a inovadora abordagem de SSMs são caminhos viáveis e promissores para este problema.

Como trabalhos futuros, propõe-se uma otimização de hiperparâmetros para as arquiteturas InceptionTime e Mamba, a fim de explorar seu pleno potencial. A aplicação destes modelos a outros conjuntos de dados, como os da missão Kepler, também seria de grande valia para testar a generalidade dos achados. Adicionalmente, a exploração de técnicas de Inteligência Artificial Explicável (XAI) é um caminho promissor para entender quais características das curvas de luz cada modelo prioriza, justificando as diferenças de comportamento observadas.

Referências

- [Astropy Collaboration et al. 2013] Astropy Collaboration, Robitaille, T. P., Tollerud, E. J., Greenfield, P., Droettboom, M., Bray, E., Aldcroft, T., Davis, M., Ginsburg, A., Price-Whelan, A. M., and et al. (2013). Astropy: A community Python package for astronomy. *Astronomy & Astrophysics*, 558:A33.
- [Borucki et al. 2010] Borucki, W. J., Koch, D., Basri, G., Batalha, N., Brown, T. M., Caldwell, D., Christensen-Dalsgaard, J., Cochran, W. D., DeVore, E., Dunham, E. W., Gautier III, T. N., Geary, J. C., Gilliland, R., Gould, A., Jenkins, J. M., Kondo, Y., Latham, D. W., Marcy, G., Meibom, S., Kjeldsen, H., Lissauer, J. J., Monet, D. G., Morrison, D., Sasselov, D., Tarter, J., Boss, A., Brownlee, D., Owen, T., Buzasi, D., Charbonneau, D., Doyle, L., Fortney, J., Holman, M. J., Seager, S., Steffen, J. H., Welsh, W. F., Rowe, J., Anderson, H., Buchhave, L. A., Ciardi, D., Walkowicz, L., Sherry, W., Horch, E., Isaacson, H., Everett, M., and Fischer, D. (2010). Kepler planet-detection mission: Introduction and first results. *Science*, 327(5968):977–980.
- [Charbonneau et al. 2000] Charbonneau, D., Brown, T. M., Latham, D. W., and Mayor, M. (2000). Detection of planetary transits across a sun-like star. *The Astrophysical Journal Letters*, 529(1):L45–L48.
- [Christ et al. 2018] Christ, M., Braun, N., Neuffer, J., and Kempa-Liehr, A. W. (2018). Time series feature extraction on basis of scalable hypothesis tests (tsfresh – a python package). *Neurocomputing*, 307:72–77.
- [Elgendy 2020] Elgendy, M. (2020). *Deep Learning for Vision Systems*. Manning Publications Co, Shelter Island.
- [Fawcett 2006] Fawcett, T. (2006). An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874.
- [Goodfellow et al. 2016] Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press. <http://www.deeplearningbook.org>.

- [Gu and Dao 2023] Gu, A. and Dao, T. (2023). Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*.
- [Ismail Fawaz et al. 2020] Ismail Fawaz, H., Lucas, B., Forestier, G., Pelletier, C., Schmidt, D. F., Weber, J., Webb, G. I., Idoumghar, L., Muller, P.-A., and Petitjean, F. (2020). InceptionTime: Finding AlexNet for time series classification. *Data Mining and Knowledge Discovery*, 34(6):1936–1962.
- [Jara-Maldonado et al. 2020] Jara-Maldonado, M., Alarcon-Aquino, V., Rosas-Romero, R., Starostenko, O., and Ramirez-Cortes, J. M. (2020). Transiting exoplanet discovery using machine learning techniques: A survey. *Earth Science Informatics*, 13:573–600.
- [Ke et al. 2017] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154.
- [Khan et al. 2017] Khan, M. S., Jenkins, S., and Yoma, N. (2017). Discovering new worlds: a review of signal processing methods for detecting exoplanets from astronomical radial velocity data. *IEEE Signal Processing Magazine*, 34:104–115.
- [Kunimoto et al. 2021] Kunimoto, M., Huang, C., Tey, E., Fong, W., Hesse, K., Shporer, A., Guerrero, N., Fausnaugh, M., Vanderspek, R., and Ricker, G. (2021). Quick-look pipeline lightcurves for 9.1 million stars observed over the first year of the tess extended mission. *Research Notes of the AAS*, 5(10):234.
- [Lecun et al. 2015] Lecun, Y., Bengio, Y., and Hinton, G. (2015). *Deep learning*, volume 521. Nature Publishing Group.
- [Lightkurve Collaboration et al. 2018] Lightkurve Collaboration, Cardoso, J. V. d. M., Hedges, C., Gully-Santiago, M., Saunders, N., Cody, A. M., Barclay, T., Hall, O., Sagar, S., Turtelboom, E., Zhang, J., Tzanidakis, A., Mighell, K., Coughlin, J., Bell, K., Berta-Thompson, Z., Williams, P., Dotson, J., and Barentsen, G. (2018). Lightkurve: A Python package for Kepler and TESS data analysis. *Astrophysics Source Code Library*.
- [Löning et al. 2019] Löning, M., Bagnall, A., Ganesh, S., Kazakov, V., Lines, J., and Király, F. J. (2019). sktime: A unified interface for machine learning with time series.
- [Malik et al. 2022] Malik, A., Moster, B. P., and Obermeier, C. (2022). Exoplanet detection using machine learning. *Monthly Notices of the Royal Astronomical Society*, 513:5505–5516.
- [Manning et al. 2008] Manning, C. D., Raghavan, P., and Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press, USA.
- [Mitchell 1997] Mitchell, T. M. (1997). *Machine Learning*. McGraw-Hill, New York, NY, USA.
- [Ricker et al. 2014] Ricker, G. R., Winn, J. N., Vanderspek, R., Latham, D. W., Bakos, G. A., Bean, J. L., Berta-Thompson, Z. K., Brown, T. M., Buchhave, L., Butler, N. R., Butler, R. P., Chaplin, W. J., Charbonneau, D., Christensen-Dalsgaard, J., Clampin, M., Deming, D., Doty, J., Lee, N. D., Dressing, C., Dunham, E. W., Endl, M., Freskin, F., Ge, J., Henning, T., Holman, M. J., Howard, A. W., Ida, S., Jenkins, J. M., Jernigan, G., Johnson, J. A., Kaltenegger, L., Kawai, N., Kjeldsen, H., Laughlin, G.,

- Levine, A. M., Lin, D., Lissauer, J. J., MacQueen, P., Marcy, G., McCullough, P. R., Morton, T. D., Narita, N., Paegert, M., Palle, E., Pepe, F., Pepper, J., Quirrenbach, A., Rinehart, S. A., Sasselov, D., Sato, B., Seager, S., Sozzetti, A., Stassun, K. G., Sullivan, P., Szentgyorgyi, A., Torres, G., Udry, S., and Villaseñor, J. (2014). Transiting exoplanet survey satellite. *Journal of Astronomical Telescopes, Instruments, and Systems*, 1:014003.
- [Samuel 1959] Samuel, A. L. (1959). Some studies in machine learning using the game of checkers. *IBM Journal of Research and Development*, 3(3):210–229.
- [Savitzky and Golay 1964] Savitzky, A. and Golay, M. J. E. (1964). Smoothing and differentiation of data by simplified least squares procedures. *Analytical chemistry*, 36(8):1627–1639.
- [Shalev-Shwartz and Ben-David 2014] Shalev-Shwartz, S. and Ben-David, S. (2014). *Understanding Machine Learning - From Theory to Algorithms*. Cambridge University Press.
- [Shallue and Vanderburg 2018] Shallue, C. J. and Vanderburg, A. (2018). Identifying exoplanets with deep learning: A five-planet resonant chain around kepler-80 and an eighth planet around kepler-90. *The Astronomical Journal*, 155:94.
- [Szegedy et al. 2015] Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., Erhan, D., Vanhoucke, V., and Rabinovich, A. (2015). Going deeper with convolutions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1–9.
- [Tey et al. 2022] Tey, E., Moldovan, D., Kunimoto, M., Huang, C. X., Shporer, A., Daylan, T., Muthukrishna, D., and Vanderburg, A. (2022). Astronet-triage-v2 dataset.
- [Tey et al. 2023] Tey, E., Moldovan, D., Kunimoto, M., Huang, C. X., Shporer, A., Daylan, T., Muthukrishna, D., Vanderburg, A., Dattilo, A., Ricker, G. R., and Seager, S. (2023). Identifying exoplanets with deep learning. v. improved light-curve classification for tess full-frame image observations. *The Astronomical Journal*, 165:95.
- [Winn 2010] Winn, J. N. (2010). Exoplanet transits and occultations. *Exoplanets*, pages 55–77.
- [Wolszczan and Frail 1992] Wolszczan, A. and Frail, D. A. (1992). A planetary system around the millisecond pulsar psr1257+12. *Nature*, 355:145–147.
- [Yu et al. 2019] Yu, L., Vanderburg, A., Huang, C., Shallue, C. J., Crossfield, I. J. M., Gaudi, B. S., Daylan, T., Dattilo, A., Armstrong, D. J., Ricker, G. R., Vanderspek, R. K., Latham, D. W., Seager, S., Dittmann, J., Doty, J. P., Glidden, A., and Quinn, S. N. (2019). Identifying exoplanets with deep learning. iii. automated triage and vetting of tess candidates. *The Astronomical Journal*, 158(1):25.